

SISTEMAS DE BANCO DE DADOS 2

AULA 4

SQL - Restrições

MySQL e ORACLE

Vandor Roberto Vilardi Rissoli



APRESENTAÇÃO

- *Alter Table*
- *Primary, Foreign Key, Unique*
- *Check e Assertion*
- *Select com algumas cláusulas*
 - *Distinct e Alias*
 - *Group by e Order by*
- *Referências*



SQL

Instrução ALTER TABLE

Por meio desta instrução é possível :

- adicionar um novo atributo;
- modificar um atributo existente;
- eliminar um atributo da relação e **muito mais...**

ALTER TABLE <tabela> [**tarefa**] (**informações da tabela**);

ALTER TABLE ESTADO

ADD (cidade VARCHAR(10)); -- inclui novo atributo na tabela

ALTER TABLE ESTADO -- altera atributo

MODIFY (cidade VARCHAR (30));

ALTER TABLE ESTADO -- apaga atributo

DROP COLUMN cidade); 

Eliminação de um atributo por vez e a relação se mantém sempre com um atributo

SQL

Restrições de Integridade

As instruções de integridade são criadas junto com a tabela ou podem ser incluídas depois da sua criação, por meio da instrução **ALTER TABLE**.

Um tipo de restrição, que já foi estudado, é a criação de um atributo obrigatório (**NOT NULL**), em que os atributos devem possuir um valor, NÃO podendo ser NULOS (**ficar sem valor**) para serem inseridos na tabela.



SQL

A identificação e a criação de uma **chave primária** consiste também em uma restrição. Esta restrição (chave primária ou ***PK***) cria um recurso relevante na relação para se estabelecer o relacionamento seguro entre relações.



Cada relação só pode possuir **uma chave primária**, no qual esta chave é formada por um atributo ou conjunto de atributos da relação (**PK simples** ou **composta**, mas é só **uma chave primária por tabela**).



SQL

Esta restrição impõe a **exclusividade** do atributo (ou conjunto de atributos para composta), além de assegurar que **nenhum** atributo desta chave seja NULO.

CONSTRAINT <nome> **PRIMARY KEY** (<atributos>);

Cria-se a chave primária junto com a criação da tabela:

```
CREATE TABLE ESTADO (  
    sigla VARCHAR(2) NOT NULL ,  
    nome VARCHAR(20) ,  
    CONSTRAINT ESTADO_PK PRIMARY KEY (sigla)) ;
```

Tabela criada.

← resultado

Por meio desta instrução foi criada uma relação de nome **ESTADO**, com uma chave primária de nome **ESTADO_PK**, que sempre terá um valor único na relação.

SQL

A criação pode ser feita para uma relação que já exista, como:

```
DROP TABLE ESTADO;           -- apaga a relação já criada
CREATE TABLE ESTADO (
    sigla VARCHAR(2) NOT NULL,
    nome VARCHAR(20) );
```

Na instrução a seguir é criada a chave primária para a relação **ESTADO**, por meio de um **ALTER TABLE**.

```
ALTER TABLE ESTADO
    ADD CONSTRAINT ESTADO_PK PRIMARY KEY (sigla);
```

Com estas instruções será criada uma relação de nome **ESTADO** que possui a chave primária **sigla**, em que esta chave primária é responsável pela identificação única de **uma tupla** na relação **ESTADO**. Veja o exemplo acima.

SQL

Após a criação da relação **ESTADO**, inicia-se o processo de construção do BD, em que os dados estarão sendo inseridos.

```
INSERT INTO ESTADO VALUES('AM', 'AMAZONAS');  
INSERT INTO ESTADO VALUES('SP','SAO PAULO');  
INSERT INTO ESTADO VALUES('SC','SANTA CATARINA');  
INSERT INTO ESTADO VALUES('AM', 'AMAPA');
```

*ERRO na linha 1:

Restrição exclusiva (**SYS.ESTADO_PK**) violada

← resultado

```
INSERT INTO ESTADO VALUES('AM', 'AMAPA');    -- Já existe
```

Como pode ser observado acima, a inserção do estado AMAPA não pode ser feita, pois por algum motivo tentou-se inserir, equivocadamente, este estado com a SIGLA de 'AM'. Este atributo é chave primária da relação **ESTADO** e já apresenta o estado AMAZONAS com esta SIGLA.

SQL

Realmente a SIGLA **AM** representa o estado do Amazonas, por isso pode-se imaginar que a inserção iria ser feita de forma errônea, por erro de digitação ou por erro no processo de coleta, ou ainda obtenção dos dados que compõe esta relação. Com o estabelecimento da restrição na relação, ela **identificou o erro** que seria cometido e não permitiu o armazenamento, até então solicitado ao SGBD.

Outra restrição importante que utiliza as chamadas **chaves candidatas**, atributo que possui valor único, mas que não foi escolhido para ser a chave primária da relação, sendo identificado como atributo de valor único também. Mas para ser implementado precisa da restrição **UNIQUE**.



SQL

A restrição **UNIQUE** (**UK**) requer que cada valor do atributo (ou conjunto de atributos porque ela também pode ser composta como a **PK**) seja exclusivo, ou seja, não se repita (duplicado), como na restrição de chave primária.

Essa restrição também é chamada de **chave exclusiva**, quando envolve um único atributo, ou **chave exclusiva composta** quando envolve mais de um atributo.

Os valores nulos são aceitos nesta restrição, a não ser que o atributo envolvido na restrição (ou os atributos) esteja(m) com a restrição **NOT NULL** descrita na sua criação (a **PK** nunca pode ser nula).



SQL

A restrição **UNIQUE** pode ser criada em nível de coluna ou tabela, porém uma chave exclusiva composta só pode ser feita no nível de tabela.

Essa restrição pode ser criada juntamente com a relação ou depois da mesma, por exemplo:

```
DROP TABLE ESTADO;           -- apagando a relação já criada
```

```
CREATE TABLE ESTADO (  
    sigla VARCHAR(2) NOT NULL,  
    nome VARCHAR(20));
```

```
ALTER TABLE ESTADO           -- Inserir restrições em relação já existente  
    ADD CONSTRAINT ESTADO_PK PRIMARY KEY (sigla),  
    ADD CONSTRAINT ESTADO_UK UNIQUE (nome);
```

→ Importante: O **ORACLE** cria um **índice** exclusivo sobre a chave exclusiva para este tipo de restrição (**UK**).

SQL

Outra restrição de integridade relacionada a criação e manipulação de chaves é a restrição de integridade referencial, que cria uma chave estrangeira em uma outra relação, ou na própria relação, fazendo o auto-relacionamento.



Por meio de uma CHAVE ESTRANGEIRA, que é formada por um ou mais atributos, é criado um relacionamento com a chave primária de uma outra relação, ou com a mesma relação (auto-relacionamento).

SQL

A relação que possui a chave primária é chamada de relação pai (ou *master*), enquanto a relação com a chave estrangeira é chamada de filho (ou *detail*). Quando existir um **auto-relacionamento** em uma relação, esta relação é pai e filho ao mesmo tempo.



Suponha que seja necessário conhecer as cidades da Federação e a qual Unidade da Federação (estado) cada uma destas cidades pertencem. Para isso será necessário armazenar dados de cada uma das cidades que formam a Base de Dados desejada.



SQL

Elaborando o projeto dessa Base será necessário para atender sua necessidade que os dados relacionados a relação **CIDADE** sejam formados pelos atributos de **nome da cidade, quantidade de habitantes, estado** que ela pertence, além de um **código** que a identifique unicamente dentro da relação (sua **PK**), formando a relação **CIDADE**.

```
CREATE TABLE CIDADE (  
    idCidade INT NOT NULL, -- PK é número inteiro  
    cidade VARCHAR(40) NOT NULL,  
    habitante BIGINT,  
    estado VARCHAR(2),  
    CONSTRAINT CIDADE_PK PRIMARY KEY (idCidade));
```

Tabela criada.

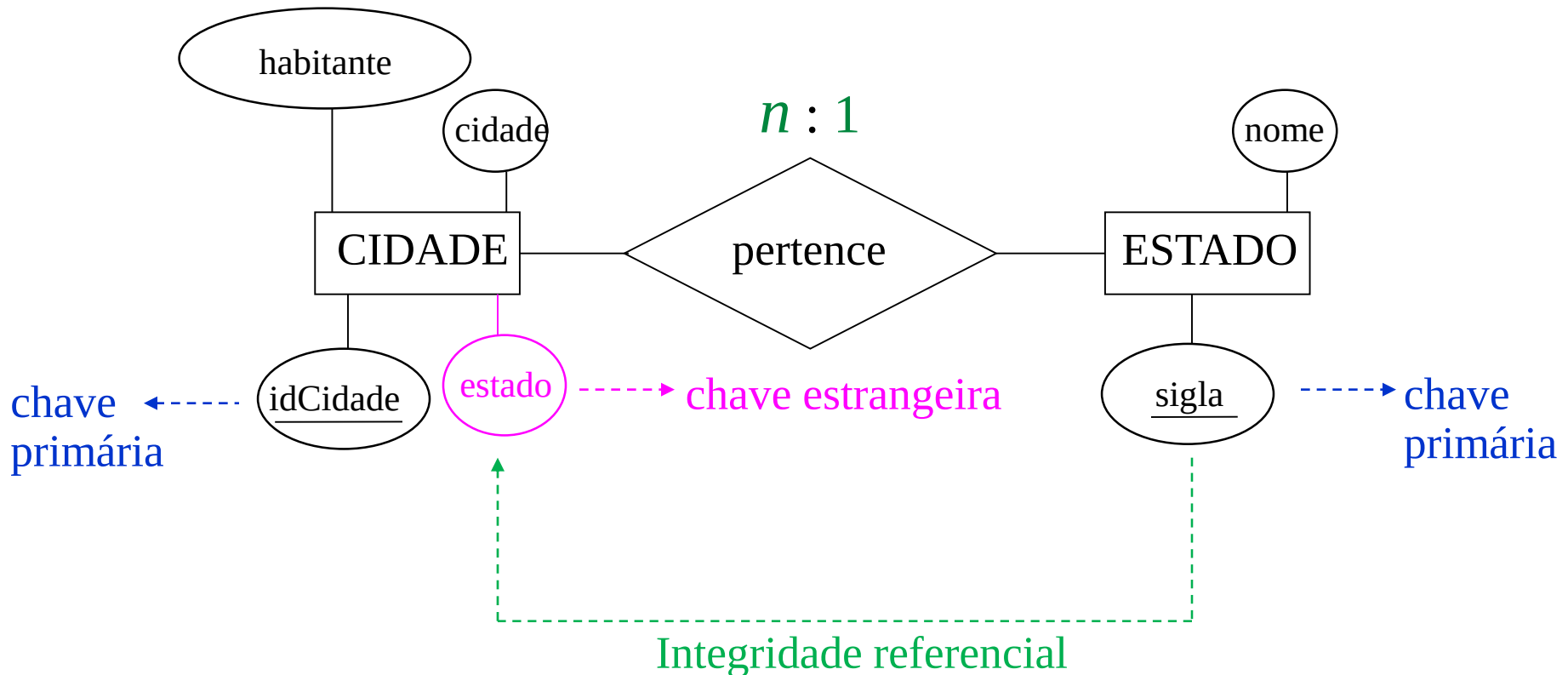


resultado



SQL

Elaborando o **ME-R** e o seu respectivo **DE-R** para esta situação tem-se abaixo o **DE-R** proposto:



O mapeamento deste modelo gerará as seguintes relações, contendo algumas restrições, tais como as seguintes:

SQL

A exemplo da chave primária, a chave estrangeira também pode ser criada juntamente com a relação ou separadamente para uma relação que já exista.

```
CREATE TABLE CIDADE (  
    idCidade INT NOT NULL,  
    cidade VARCHAR(40) NOT NULL,  
    habitante BIGINT NULL,  
    estado VARCHAR(2) NULL,  
    CONSTRAINT CIDADE_PK PRIMARY KEY (idCidade),  
    CONSTRAINT CIDADE_ESTADO_FK FOREIGN KEY (estado)  
        REFERENCES ESTADO (sigla));
```

OU

```
ALTER TABLE CIDADE  
    ADD CONSTRAINT CIDADE_ESTADO_FK  
    FOREIGN KEY (estado) REFERENCES ESTADO (sigla);
```

SQL

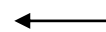
Com esta especificação de restrição uma cidade deverá possuir uma sigla de um estado já cadastrado na relação **ESTADO** (relação pai). Caso seja informada uma sigla não existente na relação pai a inserção não será realizada.

Ou a chave primária esta especificada corretamente, criando a relação entre as relações, ou ela deve ser nula na relação filho.

```
INSERT INTO CIDADE VALUES(1,'MARÍLIA',300000,'SP');  
INSERT INTO CIDADE VALUES(1,'MARÍLIA',300000,'RR');
```

* ERRO na linha 1:

Restrição de integridade (SYS.CIDADE_ESTADO_FK)
violada - chave-pai não localizada



resultado



SQL

```
INSERT INTO CIDADE VALUES(1,'MARÍLIA',300000, 'SC');
```

***ERRO na linha 1:**

← resultado

Restrição exclusiva (SYS.CIDADE_PK) violada

```
INSERT INTO CIDADE VALUES(2,'MARÍLIA',300000, null);
```

```
INSERT INTO CIDADE VALUES(3,'MARÍLIA',300000, 'SC');
```

```
SELECT * FROM CIDADE;           -- consultando os dados inseridos
```

```
IDCIDADE  NOME                QTDE_HABITANTE  SIGLA
```

-----				← resultado
1	MARÍLIA	300000	SP	
2	MARÍLIA	300000		
3	MARÍLIA	300000	SC	

```
INSERT INTO ESTADO VALUES('PR', 'PARANA');
```

```
INSERT INTO CIDADE VALUES(4,'MARINGA',400000, 'PR');
```

```
DELETE FROM CIDADE WHERE IDCIDADE = 2;
```

```
DELETE FROM CIDADE WHERE ESTADO = 'SC';
```

SQL

A chave estrangeira é sempre definida na relação filho e a relação contendo o atributo referenciado (ou atributos) é na relação pai.

Para a definição desta restrição as palavras reservadas **FOREIGN KEY**, **REFERENCES** são empregadas, podendo ainda existir **ON DELETE CASCADE**, por exemplo:

```
CREATE TABLE CIDADE (  
    idCidade INT NOT NULL,  
    cidade VARCHAR(40) NOT NULL,  
    estado VARCHAR(2) NULL,  
    CONSTRAINT CIDADE_PK PRIMARY KEY (idCidade),  
    CONSTRAINT CIDADE_ESTADO_FK FOREIGN KEY (estado)  
    REFERENCES ESTADO (sigla) ON DELETE CASCADE);
```

Identifica o(s) atributo(s)
na relação filho

Identifica a relação pai
e seu(s) atributo(s)

Identifica que as tuplas da relação filho
também serão apagadas quando a tupla da
Relação pai for apagada

SQL

As opções de restrição relacionadas as tabelas relacionadas por uma chave estrangeira no **MySQL** variam conforme a necessidade da implementação, sendo possíveis:

- CASCADE: Atualiza ou exclui os registros da tabela filho automaticamente, ao atualizar ou excluir uma tupla da pai;
- RESTRICT: Rejeita a atualização ou exclusão de um registro da tabela pai, se houver registros na tabela filho;
- SET NULL: Define como NULL o valor do campo na tabela filho, ao atualizar ou excluir o registro da tabela pai;
- NO ACTION: Equivale à opção RESTRICT, mas a verificação de integridade referencial é executada após a tentativa de alterar a tabela. É a opção PADRÃO se nenhuma opção for definida na criação de chave estrangeira;
- SET DEFAULT: Define um valor padrão para coluna da tabela filho, aplicado quando um registro da tabela pai for atualizado ou excluído.

SQL

Uma restrição **CHECK** (**CK**) especifica uma condição que deverá ser satisfeita para cada tupla da relação, podendo ela ser definida a nível de atributo ou relação.

```
CREATE TABLE CIDADE (  
    idCidade INT                NOT NULL,  
    cidade VARCHAR(40)         NOT NULL,  
    habitante BIGINT           NULL,  
    estado VARCHAR(2)          NULL,  
    CONSTRAINT CIDADE_PK PRIMARY KEY (idCidade),  
    CONSTRAINT HABITANTE_CK CHECK (habitante > 0 ));
```

Um único atributo pode possuir várias restrições **CHECK**, não havendo limite no número destas restrições que podem ser definidas em um atributo (**nível só de tupla**).



SQL

A restrição de **CHECK** pode usar as mesmas construções condicionais elaboradas nas consultas (**SELECT**), existindo algumas exceções, por exemplo em **ORACLE**:

- Não são permitidas referências às pseudocolunas CURRVAL, NEXTVAL, LEVEL e ROWNUM;

identificador inteiro do usuário corrente ←

- Nem as chamadas as funções SYSDATE, **UID**,
← retorna o nome do usuário corrente no BD USER e USERENV; → retorna informações de uma sessão corrente

- Nem as consultas que se referem a outros valores em outras tuplas... mas para isso existem alternativas, entre elas trabalhar com

ASSERTION (conteúdo estudado mais a frente)



SQL

Uma outra expressão que pode ser usada na criação de relações é a **DEFAULT** que insere um valor no atributo quando este não for informado. Observe o exemplo:

```
CREATE TABLE CIDADE (  
    idCidade          INT          NOT NULL,  
    cidade            VARCHAR(40)  NOT NULL,  
    aniversario       DATE DEFAULT SYSDATE NOT NULL ,  
    habitante         BIGINT       NULL,  
    estado            VARCHAR(2)   NULL,  
    CONSTRAINT CIDADE_PK PRIMARY KEY (idCidade),  
    CONSTRAINT HABITANTE_CK CHECK (habitante > 0 ));
```

Não precisa ser
NOT NULL

↑

SYSDATE → função que “captura” a data do sistema (**SO**) em **ORACLE**.

```
INSERT INTO CIDADE VALUES(1, 'Brasília', '20-apr-2001', 2000000);  
INSERT INTO CIDADE (idCidade, cidade, habitante)  
VALUES (2, 'São Paulo', 4000000);
```

↗

SQL

O **MySQL** passou a implementar completamente na versão 8 a restrição de **CHECK**.

Observando o uso do SYSDATE no **ORACLE** é interessante também lembrar desse recurso no **MySQL**, em que as funções CURDATE e NOW são recursos similares.

CURDATE() => fornece só a data no padrão AAAA-MM-DD

AAAA => ano com 4 dígitos

MM => mês com 2 dígitos

DD => dia com 2 dígitos

NOW() => fornece data e horário no padrão

AAAA-MM-DD **HH:MM:SS**

HH => hora com 2 dígitos

MM => minuto com 2 dígitos

SS => segundo com 2 dígitos

Consultando o Banco de Dados

Elabore uma consulta que permita visualizar todos os dados do esquema **ESTADO** e **CIDADE**.

Para que todos os dados das 2 relações sejam apresentados é necessário realizar a operação de **junção** entre elas.

Apesar de existir o relacionamento entre as 2 relações, este relacionamento só será usado na apresentação dos dados (via **SELECT** por exemplo), somente se for especificada a expressão de **junção** na consulta, por exemplo:

```
SELECT nome, sigla FROM ESTADO;
```

```
SELECT idCidade, cidade, aniversario, habitante  
FROM CIDADE;
```

Estas consultas apresentam todas as tuplas de cada relação, porém a demanda acima deseja a apresentação de todos estes dados já relacionados em **uma só consulta**.

SQL

Para isso se utiliza a **junção** entre as relações, que podem ser feitas para mais que 2 relações, caso exista esta necessidade.

```
SELECT nome. sigla, idCidade, cidade, habitante, estado  
FROM ESTADO, CIDADE -- consulta sobre 2 relações  
WHERE sigla = estado; -- realizando a junção entre as duas
```

Apresentando os dados de maneira mais organizada, pode-se classificá-los como for mais interessante, por exemplo, separar por **sigla** primeiro e dentro das siglas por uma ordenação alfabética do nome da cidade.

```
SELECT nome, sigla, idCidade, cidade, habitante, estado
FROM ESTADO, CIDADE
WHERE sigla = estado
ORDER BY sigla, cidade;
```

SQL

O resultado padrão a ser apresentado por uma instrução *SELECT* exibi todos os atributos solicitados em todas as tuplas selecionadas, o que pode incluir tuplas duplicadas.

Caso isso não seja desejável por uma determinada consulta, o qualificador **DISTINCT** deve ser inserido na instrução *SELECT* que será executada. Este qualificador vem logo após a palavra-reservada *SELECT*, onde em seguida podem ser especificados todos os atributos que se deseja apresentar como resultado único por linha desta consulta.

Por exemplo: suponha que a relação **CIDADE** possua 6 tuplas, sendo 3 cidades do estado de São Paulo (SP).

```
SELECT sigla  
FROM root.CIDADE  
WHERE sigla = 'SP';
```

Esta consulta selecionará somente as cidades com a sigla do estado de São Paulo (SP), mostrando assim **3 tuplas idênticas**.

SQL

Este tipo de consulta também poderia incluir o qualificador **ALL** no início do *SELECT*, porém esta qualificação já é padrão quando nenhum qualificador é especificado, pois ela estabelece que todas (**ALL**) as tuplas selecionadas devem ser apresentadas.

```
SELECT ALL sigla  
FROM root.CIDADE  
WHERE sigla = 'SP';
```

Para que os valores selecionados não sejam apresentados em duplicidade o qualificador **DISTINCT** deve ser colocado no local do **ALL**, não mostrando algo desagradável ao usuário (**várias tuplas idênticas mostradas**).

```
SELECT DISTINCT sigla  
FROM root.CIDADE  
WHERE sigla = 'SP';
```

Esta consulta apresentará uma única tupla com o valor da sigla SP, embora várias foram recuperadas.

SQL

Quando os atributos solicitados não formarem uma tupla **totalmente** única eles serão apresentados normalmente.

```
SELECT DISTINCT sigla, codigo  
FROM CIDADE  
WHERE sigla = 'SP';
```

O resultado apresentado consistirá de todas as tuplas que possuem a **sigla** SP, com códigos diferentes, pois se existir alguma sigla igual com código igual o qualificador **DISTINCT** não permitirá a sua apresentação como resultado dessa consulta.



SQL

FUNÇÕES DE GRUPO

As funções de grupo trabalham com conjunto(s) de tuplas, ao invés de uma única tupla por vez, fornecendo o resultado do *SELECT* também avaliado por grupo.

As funções de grupo padrões presentes no SQL são:

AVG - valor médio do grupo, ignorando os atributos nulos

COUNT - conta quantidade de tuplas (* - inclui nulos e duplicat.)

MAX - valor máximo ignorando o nulo

MIN - valor mínimo ignorando o nulo

STDDEV - desvio padrão dos valores selecionados, sem nulos

SUM - valores somados ignorando os nulos

VARIANCE - variação dos valores selecionados, sem nulos



SQL

As funções de grupo **ignoram valores nulos** quando o atributo for usado para agrupar valores.

A exceção é a função **COUNT** que pode ser usada com o parâmetro pré-definido ***** (asterisco) que solicitará a quantidade de tuplas que existe na relação referenciada, inclusive as com valor nulo.

```
SELECT COUNT(*)  
FROM CIDADE;
```

Todas as demais funções de grupo **ignoram os valores nulos**, porém estas podem fazer uso também da função **NVL()**, caso seja necessário trabalhar com nulos no ambiente **ORACLE**.

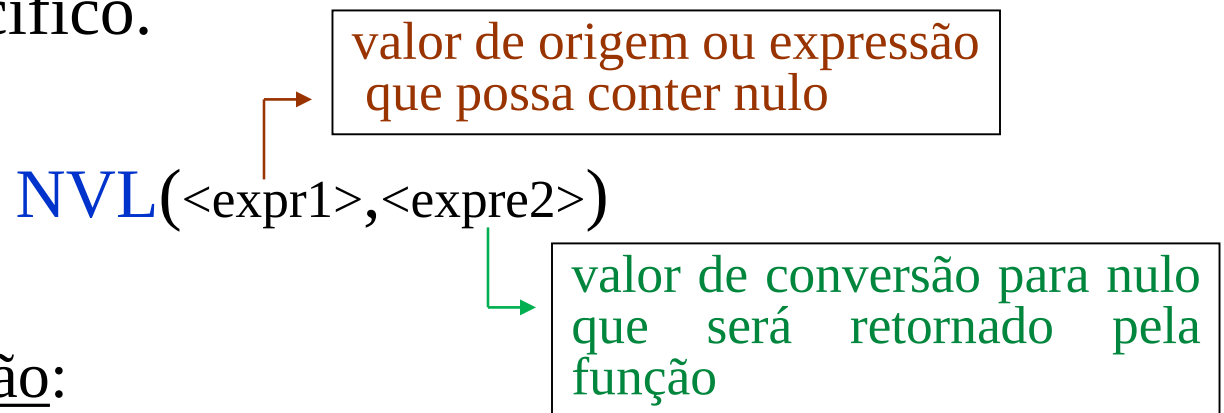


SQL

FUNÇÃO NVL()

Esta função permite o tratamento de atributos que **possuam o valor nulo armazenado**.

Ela pode ser aplicada sobre qualquer tipo de dado (numérico, caracter, data), onde o atributo que possuir seu valor nulo armazenado retornará um outro dado definido para este tipo específico.



Exemplo de conversão:

NUMÉRICO = NVL(<coluna numérica>, 0)

DATA = NVL(<coluna de data>, '01-JAN-95')

CARACTER = NVL(<coluna caracter>, 'indisponível')

SQL

No **MySQL** existem algumas possibilidades para o tratamento de valores nulos em um atributo, quando for necessário.

IFNULL – função com 2 parâmetros para tratar nulo

IFNULL(<atributo>, <valor>)

<atributo> => atributo da tabela que pode ter valor NULL;

<valor> => valor numérico ou caracteres que substituirá o valor NULL, quando recuperado da tabela;

Exemplos: **IFNULL**(telefone, 'Sem telefone')
 IFNULL(salario, 0)

→ Função versátil que pode ser aplicada no *SELECT*, *WHERE*, *GROUP BY* e *ORDER BY*.

SQL

Uma outra função proveniente do padrão SQL (ANSI) que possibilita o tratamento de valores nulos é a **COALESCE** (disponível em vários SGBD relacionais como **MySQL**, **PostgreSQL**, **ORACLE**, SQL Server, etc.).

Essa função possui vários parâmetros e retorna o **primeiro valor** da esquerda para direita que **não for nulo**.

Exemplos:

COALESCE (telefone, celular, 'Sem telefone')

COALESCE (salario, 0)

→ Mas atenção porque se todos os valores de seus parâmetros forem nulos a **COALESCE** retornará nulo (**NULL**).



SQL

CRIANDO GRUPOS DE DADOS

Tudo que foi feito até aqui tratava todos os dados de uma relação como um único grupo de dados, mas as vezes é necessário dividir estas relações em grupos menores.

Procurando realizar estas divisões nos dados é que se pode incluir a cláusula **GROUP BY** em uma instrução *SELECT*.

É possível aplicar uma cláusula **GROUP BY** em uma seleção e usar uma função de grupo para **sumarizar** os valores de cada um dos grupos formados durante a consulta.



SQL

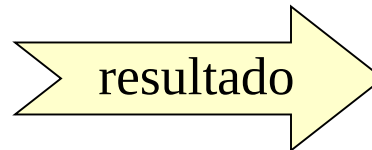
Suponha que a consulta abaixo seja executada e o resultado seja o apresentado a seguir.

O que isso significa?

O que isso quer dizer?

A partir daqui as instruções SQL mostradas são para solução do *slide* 27.

```
SELECT sigla, COUNT(sigla) AS Conta  
FROM CIDADE  
GROUP BY sigla;
```



sigla	Conta
-----	-----
CE	7
DF	3
SP	14

A relação **CIDADE** foi dividida em 3 grupos por meio da sigla, onde é considerada somente as siglas que estão cadastradas e são apresentadas por meio da seleção do atributo **sigla**, enquanto que a função **COUNT**, sobre a **sigla**, mostra a quantidade de tuplas que existe em cada **sigla** (alguns grupos).

SQL

Sempre que existir um ou mais atributos individuais na seleção, juntamente com a aplicação de funções de grupo, será necessário incluir estes atributos no grupo por meio da cláusula **GROUP BY**.

```
SELECT sigla, COUNT(sigla)
```

```
FROM CIDADE
```

```
GROUP BY sigla; → um atributo simples ou comum
```

A seleção sem especificação de nenhum atributo individual também é possível, mas será realizada por toda a relação se não for especificada a cláusula **GROUP BY**. Observe os resultados em seu BD aplicando as duas sugestões a seguir.

```
SELECT COUNT(sigla)
```

```
FROM CIDADE
```

```
GROUP BY sigla;
```

≠

```
SELECT COUNT(sigla)
```

```
FROM CIDADE;
```

SQL

O atributo especificado no **GROUP BY** não precisa estar no *SELECT*, mas normalmente seu resultado fica sem sentido devido a dificuldade de compreensão dos dados agrupados por quem vai visualizar o resultado.

```
SELECT COUNT(sigla)
FROM CIDADE
GROUP BY sigla;
```

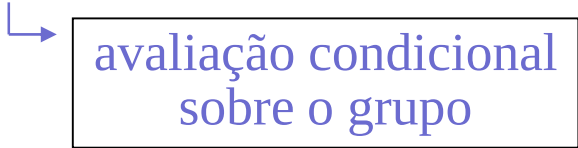
Pode também existir a necessidade de agrupar mais que um atributo, em que uma relação é agrupada primeiro por um atributo e dentro deste atributo também é realizado o agrupamento por um outro atributo. Suponha o exemplo:

```
SELECT departamento, empregado, SUM(salario)
FROM EMPREGADO
GROUP BY departamento, empregado;
```

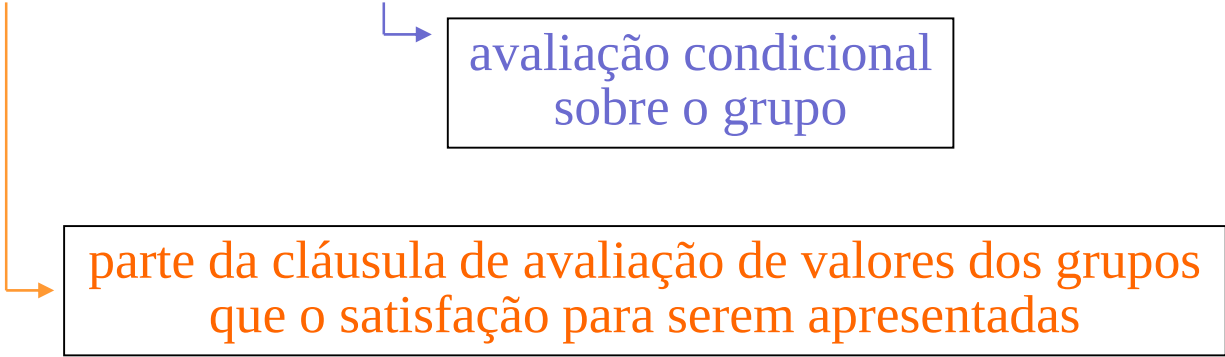
SQL

O uso da cláusula **WHERE** não pode ser aplicada na restrição (filtro) de grupos, mas a inclusão da cláusula **HAVING** pode avaliar cada grupo e permitir a sua apresentação ou não.

```
SELECT departamento, AVG(salario) AS media  
FROM EMPREGADO  
GROUP BY departamento  
HAVING media > 2000;
```



avaliação condicional
sobre o grupo



parte da cláusula de avaliação de valores dos grupos
que o satisfação para serem apresentadas

→ Como é usada a cláusula **WHERE** na restrição de tuplas sobre atributos individuais, a cláusula **HAVING** também pode ser empregada, mas só sobre funções de grupo da consulta.

SQL

Pode ser usada a cláusula **GROUP BY** sem uma função de grupo no *SELECT*. Porém, a apresentação dos resultados sempre acontecerão em ordem crescente quando um cláusula de grupo estiver na instrução.

Um **ORDER BY** pode ser inserido, mas por padrão a apresentação sempre será em ordem crescente. A aplicação do **ORDER BY** seria coerente para a alteração desta apresentação, pois o resultado desejado seria melhor realizado em uma ordenação decrescente, por exemplo, ou mesmo sobre outros atributos da consulta que deseje o usuário.

```
SELECT departamento, AVG(salario) AS media  
FROM EMPREGADO  
GROUP BY departamento  
      HAVING media > 2000  
ORDER BY departamento;
```

SQL

É importante esclarecer que NÃO existe cláusula **HAVING** em uma consulta SQL, como não existe **FROM** também como cláusula. O **FROM** é parte obrigatória da instrução **SELECT**.

Assim, **HAVING** é parte opcional da cláusula GROUP BY, bem como a cláusula **WHERE** é opcional do **SELECT**.

```
SELECT departamento, AVG(salario) AS media  
FROM EMPREGADO  
HAVING media > 2000);
```

↳ avaliação condicional
sobre o grupo

parte da cláusula de agrupamento que nem existe na consulta
(cadê o GROUP BY?) - **ERRO**



Exercícios de Fixação

- 1.) Elabore um conjunto de relações que permita armazenar e consultar os dados de alunos matriculados em um curso. Para isso, identifique quais são os dados necessários sobre o aluno e o curso, elaborando o **ME-R** (no modelo) e seu respectivo **DE-R** proposto. Em seguida, realize o mapeamento do **DE-R** para as relações que formarão a base de dados com as restrições estudadas até o momento. Implemente no ambiente SQL e insira alguns dados para testá-los. Por fim, elabore 3 consultas:
- a) todos os alunos matriculados;
 - b) todos os cursos disponíveis na instituição;
 - c) todos os alunos com a identificação completa do curso que ele esta cursando.

Exercícios de Fixação

2.) Usando o **modelo ME-R** padrão, elabore um projeto que implemente um BD para o controle de funcionários de uma empresa. Este controle visa gerenciar as atividades de cada funcionário que trabalha em um departamento específico dentro de uma única empresa. O motivo deste controle é reconhecer o envolvimento de cada departamento nos diferentes projetos que a empresa realiza, identificando com isso os departamentos envolvidos em cada projeto e os funcionários destes departamentos que fazem parte do projeto pesquisado. Os dados disponíveis dos projetos são sempre o nome, a data prevista de início e de fim do projeto, além do funcionário responsável pelo projeto. Os departamentos possuem nome, além da identificação dos funcionários que o compõe.

Exercícios de Fixação

... continuação do exercício 2

Os funcionários são contratados após a coleta dos dados pessoais (nome, CPF, carteira profissional, telefones e endereço residencial) de cada um, além da identificação do cargo que ele estará ocupando, seu salário mensal e o departamento ao qual estará alocado. Faça também as seguintes consultas para atender o que esta empresa precisa:

- a)** Identificação nominal do funcionário e seu endereço para uso do serviço de mala direta (etiqueta para correio);
- b)** Os funcionários que pertencem a um departamento;
- c)** Relatório de custos separado por departamento identificando os salários dos funcionários que formam tal departamento;
- d)** Os departamentos envolvidos em um projeto;
- e)** Os funcionários envolvidos em um projeto.

FUNÇÕES DE JANELA

HISTÓRICO

O SGBD **Oracle 8i** em 1998 introduziu processamento interessante nos resultados mostrados em suas consultas, sendo integrado ao padrão SQL cinco anos depois como **Funções de Janela**, a partir do **SQL 2003**.

- Opera sobre a "**janela de dados**" que está sendo processada pela consulta (*SELECT*) em execução;
- Sua cláusula *OVER()* define esta "**janela de dados**";
- Possibilita calcular somas cumulativas, médias móveis ou faz classificações organizadas dentro do **contexto** de uma tupla (**linha**) em processamento.



FUNÇÕES DE JANELA

Ela corresponde a uma **função** que executa um cálculo sobre um conjunto de tuplas relacionadas à **tupla atual**, SEM agrupá-las em uma única saída, como acontece nas funções de agregação SQL.

➤ Cláusula **OVER()**:

- É obrigatória e define qual a **janela** que a função irá operar;

➤ Opção **PARTITION BY**:

- Em **OVER()**, esta opção pode existir e dividirá os dados em grupos ou partições, permitindo que o cálculo seja feito separadamente para cada grupo;



FUNÇÕES DE JANELA

➤ Opção **ORDER BY**:

- Em **OVER()**, esta opção pode existir e ordenará as tuplas que estão sendo processadas dentro de **cada partição**, sendo crucial para funções que dependem de uma sequência, como nas somas acumulativas.

Exemplo:

ALBUM (idAlbum, titulo, artista, copiasVendida, periodoVenda)

SELECT periodoVenda, titulo, artista, copiasVendida,
 SUM(copiasVendida)

OVER(ORDER BY periodoVenda ASC) **AS** **acumulaSoma**

FROM **ALBUM**

WHERE titulo = 'In the Right Place';



FUNÇÕES DE JANELA

Resultado (*data set*)

Ordem crescente ↓

periodoVenda	titulo	artista	copiasVendida	acumulaSoma
2022_1Q	In the Right Place	Dr. John	222	222
2022_2Q	In the Right Place	Dr. John	208	430
2022_3Q	In the Right Place	Dr. John	94	524
2022_4Q	In the Right Place	Dr. John	912	1436
2023_1Q	In the Right Place	Dr. John	912	2348
2023_2Q	In the Right Place	Dr. John	56	2404
2023_3Q	In the Right Place	Dr. John	562	2966

Vendas 4º trimestre

Vendas acumula anteriores

FUNÇÕES DE JANELA

Diferença entre Funções de Janela e de Agregação

As **Funções de Janela** **não** agrupam dados, compactando os valores contidos em várias tuplas em um único valor, mas retornam um valor para cada tupla da tabela que estará sendo mostrada, sendo este valor resultado de um cálculo que opera o grupo de tuplas que está na "**janela de dados**".

- Para observar mais exemplos iniciais neste estudo consulte o sítio virtual:

<https://learnsql.com.br/blog/o-que-sao-funcoes-de-janela-do-sql/>



Exercícios de Fixação

3.) Crie uma consulta SQL que apresente o identificador de Pesquisa, a descrição da Pesquisa e o identificador da Pergunta, além de uma **Função de Janela** que irá contar acumuladamente pelo identificador da pesquisa. Mas para uma nova pesquisa este acumulador, chamado **Contador**, sempre começará novamente em 1 a sua contagem acumulativa.

Sua consulta deverá pesquisar agrupando pelos identificadores da pesquisa e da pergunta. A ordem da apresentação será **crescente** do identificador da pesquisa.

Para encerrar este exercício será necessário realizar uma pesquisa de como mostrado o CUSTO da execução desta consulta por meio de uma única instrução do **MySQL**, logo após a consulta ser executada.

Exercícios de Fixação

... continuação do exercício 3

O valor exato desse CUSTO de execução pelo SGBD deverá ser mostrado em uma **tabela** com 3 execuções em seguida, contendo ainda o tempo de execução em segundos (**sec.**) para cada uma das 3 execuções.

Essa **tabela** será apresentada como comentário no *script* logo após a consulta em SQL e a instrução que apura o CUSTO no *script* denominado **Consulta** com a documentação exigida em TODOS *scripts* feitos nesta disciplina (exemplo ilustrativo abaixo da tabela com dados).

Execução	Custo	Tempo (sec.)
1	2345.09	0.095
2	2345.09	0.041
3	2345.09	0.041

Exercícios de Fixação

... continuação do exercício 3

Sem a documentação exigida na disciplina para qualquer *script* ele não é considerado elaborado a contento da disciplina, independente do que exista escrito nele.

Uma **dica** importante é que o CUSTO e o TEMPO devem ser coletados imediatamente após cada execução dessa consulta que resolve este exercício, sendo este CUSTO apurado e mostrado no **MySQL** por **uma só instrução** SHOW da última consulta que acabou de ser executada por este SGBD.

Acesse a base de dados que será indicada por seu professor para realizar mais este exercício prático.

Referência de Criação e Apoio ao Estudo

Material para Consulta e Apoio ao Conteúdo

- ELMASRI, R. e Navathe, S. B., Fundamentals of Database Systems, Addison-Wesley, 3rd edition, 2000
 - Capítulo 8
- SILBERSCHATZ, A. & Korth, H. F., Sistemas de Banco de Dados.
 - Capítulo 4
- SUNDERRAMAN, R., Oracle Programming: A Primer, Addison Wesley, 1999.
 - Capítulo 2
- Oracle: *Window Functions*
 - https://docs.oracle.com/cd/E17952_01/mysql-8.0-en/window-functions.html



Referência de Criação e Apoio ao Estudo

Material para Consulta e Apoio ao Conteúdo (continuação)

- Universidade de Brasília (UnB Gama - **CAE**)
 - <https://sae.unb.br/cae/conteudo/unbfga>
(escolha a disciplina **Sistemas de Banco de Dados 1**)
- Oracle: SQL Assertions/Declarative multi-row constraints
 - <https://community.oracle.com/ideas/13028>

